

1.Metryka projektu	1
2.Architektura środowiska sieciowego	1
2.1 Opis maszyn.....	1
3.Opis wdrożenia	2
4.Weryfikacja i testy	4
4.1 Brute Force: Użycie Hydra z Kali Linux, aby zaatakować SSH monitorowanej maszyny	4
4.2 Web Attack: Zmodyfikowanie treści strony głównej (symulacja włamania).	6
5.Wnioski i rekomendacje.....	7
5.1. Czy rozwiązanie jest skuteczne?	8
5.2. Trudności napotkane podczas wdrożenia	8
5.3. Wymagania do wdrożenia w skali całej firmy (Skalowalność).....	8

1.Metryka projektu

Wybrana ścieżka: Ścieżka C - Monitoring

Użyte technologie: VMware Fusion, macOS

Data realizacji: 11.04.2026

2.Architektura środowiska sieciowego

2.1 Opis maszyn

Maszyna 1: Serwer Centralny (Wazuh Manager)

- **System operacyjny:** macOS (architektura Apple Silicon / ARM64)
- **Środowisko wdrożeniowe:** Docker (wdrożenie kontenerowe typu Single-node)
- **Rola w projekcie:** Pełniła funkcję głównej centrali analitycznej (SIEM/XDR). Odbierała szyfrowane logi z agentów w czasie rzeczywistym (w modelu komunikacji PUSH), analizowała je pod kątem złamania reguł bezpieczeństwa

oraz wyświetlała alerty w graficznym panelu analitycznym. Zarządzała również polityką automatycznej obrony (Active Response), koordynując blokady sieciowe.

Maszyna 2: Serwer Monitorowany / Cel Ataku (Wazuh Agent)

- **System operacyjny:** Ubuntu Linux 24.04 LTS (Maszyna Wirtualna, architektura aarch64)
- **Adres IP (lokalny):** 192.168.65.145
- **Kluczowe usługi:** Serwer WWW (Nginx), demon SSH (sshd)
- **Rola w projekcie:** Wcielała się w rolę produkcyjnego serwera ofiary. Działał na niej "Wazuh Agent" nadający uprawnienia audytowe. Agent monitorował próby logowania oraz śledził integralność plików w czasie rzeczywistym (moduł FIM chroniący katalog /var/www/html/). Agent wykonywał także fizyczne polecenia odcięć na lokalnym firewallu (skrypt firewall-drop).

Maszyna 3: System Atakujący (Red Team)

- **System operacyjny:** Kali Linux (Maszyna Wirtualna)
- **Wykorzystane narzędzia:** Powłoka Bash, hydra (narzędzie do łamania haseł online)
- **Rola w projekcie:** Służyła do symulacji działań wrogiego aktora (Adversary). Z poziomu tej maszyny przeprowadzono kontrolowane ataki na infrastrukturę monitorowaną, w tym bezpośrednią modyfikację pliku źródłowego serwera WWW (symulacja ataku Defacement) oraz zmasowany atak siłowy (Brute Force) na usługę SSH z wykorzystaniem dedykowanego słownika haseł.

3.Opis wdrożenia

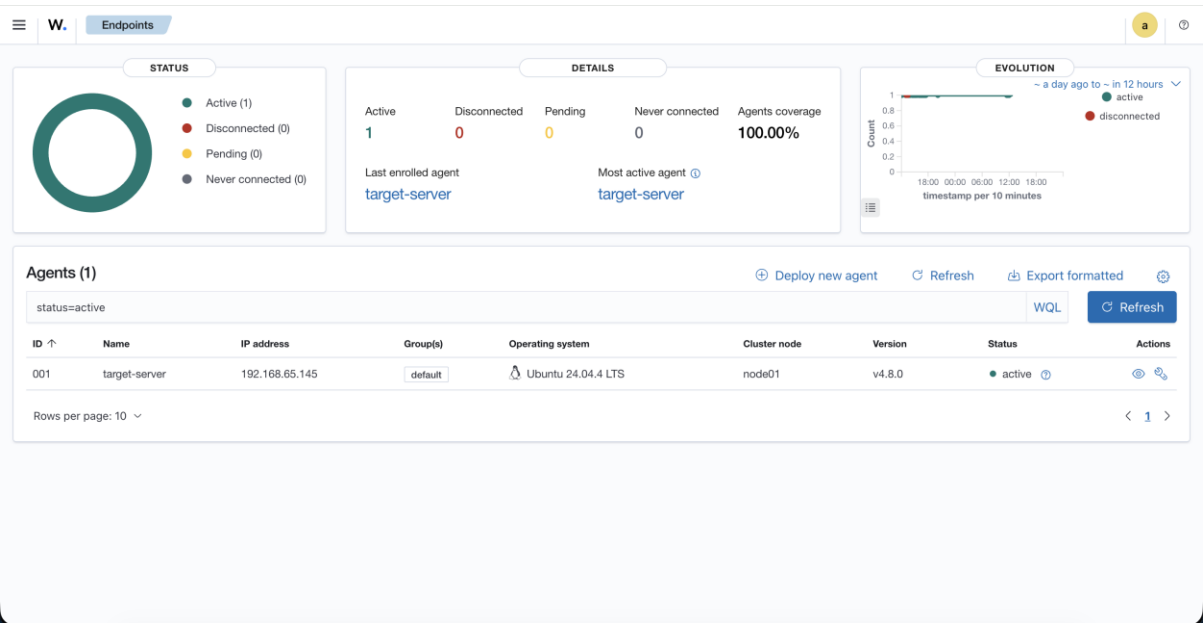
Proces budowy oraz konfiguracji hybrydowego środowiska bezpieczeństwa został podzielony na cztery główne etapy, realizowane w układzie logicznym od centrali analitycznej, przez agenta, aż po polityki obronne:

Etap 1: Wdrożenie Centrali Analitycznej (Wazuh Manager)

Pierwszym krokiem było uruchomienie serwera zarządzającego na systemie hosta (macOS). Ze względu na specyfikę architektury ARM, wykorzystano środowisko Docker do konteneryzacji usług Wazuha (Single-node deployment). Po pomyślnym pobraniu i uruchomieniu obrazów z autoryzowanego repozytorium, zweryfikowano poprawność działania konsoli webowej oraz upewniono się, że porty nasłuchujące są otwarte dla maszyn wirtualnych w sieci lokalnej.

Etap 2: Rejestracja Agentów i Zabezpieczenie Komunikacji

Na monitorowanej maszynie wirtualnej (Ubuntu) wygenerowano dedykowany pakiet instalacyjny agenta Wazuh w formacie .deb (architektura aarch64). Podczas instalacji, w pliku konfiguracyjnym agenta (/var/ossec/etc/ossec.conf) wskazano docelowy adres IP centrali, wymuszając komunikację w modelu PUSH. Po restarcie demona systemowego autoryzowano agenta, co potwierdził status "Active" w panelu głównym (Dashboard).



Etap 3: Implementacja Monitorowania Integralności Plików (FIM)

W celu wykrywania nieautoryzowanych modyfikacji w obrębie usług webowych (symulacja ochrony przed atakiem typu Defacement), zainstalowano serwer WWW Nginx. Następnie zmodyfikowano plik konfiguracyjny agenta na systemie Ubuntu, dodając dyrektywę `<directories realtime="yes">/var/www/html</directories>`. Parametr `realtime` wymusił ciągłe monitorowanie krytycznego katalogu bez opóźnień wynikających z cyklicznego skanowania, opierając się na natychmiastowym przeliczaniu skrótów kryptograficznych (hash) plików.

Etap 4: Konfiguracja Zautomatyzowanej Obrony (Active Response)

Finałowym etapem wdrożenia było przejście z modelu pasywnego (wyłącznie alertowanie) na model aktywny (IPS/XDR). W głównym pliku konfiguracyjnym serwera centralnego (`ossec.conf`) zdefiniowano nową regułę sekcji `<active-response>`. Powiązano w niej detekcję ataku siłowego na usługę SSH (Rule ID: 5763) ze skryptem `firewall-drop` egzekwowanym lokalnie na zaatakowanym agencie. Czas przewencyjnej blokady adresu IP atakującego (`timeout`) ustalono na 60 sekund. Po restarcie menedżera reguł, system uzyskał zdolność do autonomicznego odpierania ataków.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>60</timeout>
</active-response>
```

4. Weryfikacja i testy

4.1 Brute Force: Użycie Hydra z Kali Linux, aby zaatakować SSH monitorowanej maszyny

Faza 1: Przygotowanie wektora ataku (Red Team)

Do przeprowadzenia symulacji wykorzystano maszynę z systemem Kali Linux. Jako główne narzędzie ofensywne wybrano program hydra, służący do szybkiego testowania danych uwierzytelniających w protokołach sieciowych. W ramach przygotowań utworzono dedykowany słownik haseł (hasla.txt) zawierający ponad 100 niepoprawnych kombinacji, w celu wymuszenia na systemie długotrwałej pracy pod obciążeniem. Atak zainicjowano komendą kierującą ruch bezpośrednio na port SSH maszyny monitorowanej (Ubuntu): `hydra -l konrad -P hasla.txt ssh://192.168.65.145`

Faza 2: Detekcja Zagrożenia

Natychmiast po rozpoczęciu ataku, agent Wazuh zainstalowany na serwerze Ubuntu zaczął monitorować napływające błędy w logach autoryzacyjnych (/var/log/auth.log) i przysyłać je do centrali. System analityczny poprawnie skorelował ze sobą pojedyncze błędy logowania (alerty poziomu 5), identyfikując złośliwy schemat działania. W ułamku sekundy wygenerowano alert krytyczny **Level 10 (Rule ID: 5763)** z sygnaturą: *"sshd: brute force trying to get access to the system"*.

>	Apr 11, 2026 @ 11:44:37.583	sshd: brute force trying to get access to the system. Authentication failed.	10	5763
>	Apr 11, 2026 @ 11:44:37.580	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.577	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.574	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.572	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.570	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.569	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 11:44:37.569	sshd: authentication failed.	5	5760

Faza 3: Automatyczna Mitygacja (Active Response / IPS)

Zgodnie z wdrożoną wcześniej polityką bezpieczeństwa, wyzwolenie reguły 5763 uruchomiło procedurę Active Response. Centrala (Wazuh Manager) wysłała do zaatakowanego agenta żądanie natychmiastowej izolacji sieciowej. Agent wykonał lokalny skrypt firewall-drop, który dodał adres IP maszyny atakującej (Kali Linux) do czarnej listy na zaporze sieciowej (iptables) na równe 60 sekund. Z perspektywy atakującego, połączenie z serwerem ofiary zostało nagle zerwane w trakcie trwania ataku. Narzędzie hydra przerwało pracę, raportując błędy sieciowe: `[ERROR] all children were disabled due too many connection errors` oraz `[ERROR] 1 target was disabled`.

```
(konrad@kali)-[~]
└─$ hydra -i konrad -P hasla.txt ssh://192.168.65.145
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-11 11:49:18
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 121 login tries (l:1/p:121), ~8 tries per task
[DATA] attacking ssh://192.168.65.145:22/
[STATUS] 45.00 tries/min, 45 tries in 00:01h, 78 to do in 00:02h, 14 active
[STATUS] 42.50 tries/min, 85 tries in 00:02h, 38 to do in 00:01h, 14 active
[STATUS] 28.33 tries/min, 85 tries in 00:03h, 46 to do in 00:02h, 6 active
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-11 11:52:23
```

>	Apr 11, 2026 @ 12:24:31.136	Host Unblocked by firewall-drop Active Response	3	652
>	Apr 11, 2026 @ 12:23:31.091	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.089	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.086	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.084	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.082	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.080	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.078	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.075	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.072	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.071	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:31.071	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:29.127	Host Blocked by firewall-drop Active Response	3	651
>	Apr 11, 2026 @ 12:23:29.117	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:29.116	sshd: authentication failed.	5	5760
>	Apr 11, 2026 @ 12:23:29.115	sshd: authentication failed.	5	5760

4.2 Web Attack: Zmodyfikowanie treści strony głównej (symulacja włamania).

Faza 1: Przygotowanie i realizacja ataku (Defacement)

Atak polegał na nieautoryzowanej podmianie oficjalnej strony głównej serwera na własny komunikat (hakerski manifest). Do przeprowadzenia symulacji wykorzystano bezpośredni dostęp do powłoki maszyny docelowej (Ubuntu). Brutalne nadpisanie oryginalnego pliku serwera Nginx (/var/www/html/index.html) zrealizowano za pomocą pojedynczego polecenia systemowego omijającego standardowe procedury edycji: `echo "<html style='background-color:black; color:red;'><center><h1>HACKED BY KONRAD v2</h1><p>Tym razem centrala patrzy!</p></center></html>" | sudo tee /var/www/html/index.html`

Faza 2: Detekcja w czasie rzeczywistym (Real-time FIM)

Dzięki zaimplementowanej wcześniej polityce monitorowania w pliku `ossec.conf` (dyrektywa `<directories realtime="yes">/var/www/html</directories>`), agent Wazuh bez przerwy nadzorował wskazany katalog pod kątem zmian kryptograficznych. W ułamku sekundy po nadpisaniu pliku `index.html`, agent wykrył drastyczną zmianę sumy kontrolnej (hash) pliku oraz jego atrybutów. Sygnał o naruszeniu integralności został natychmiast przekazany do centrali SOC na systemie macOS.

Faza 3: Analiza logów w Centrali (Wazuh Manager)

W panelu analitycznym Wazuh, w dedykowanej sekcji *File Integrity Monitoring*, automatycznie wygenerowano powiadomienie o incydencie. System prawidłowo zarejestrował próbę ataku, prezentując na wykresach dominującą akcję "Modified".

Apr 11, 2026 @ 12:38:46.990 Integrity checksum changed.

7550

Expanded document

View surrounding documents

View single document

Table

JSON

_index	wazuh-alerts-4.x-2026.04.11
agent.id	001
agent.ip	192.168.65.145
agent.name	target-server
decoder.name	syscheck_integrity_changed
full_log	File '/var/www/html/index.html' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '159' to '124' Old modification time was: '1775903825', now it is '1775903927' Old md5sum was: 'f45ab6e3e8c778a37dc2174a0a2004ab' New md5sum is: 'eb14247e394264fdbbedac4e0f9a21e3' Old sha1sum was: '74acd059387d684d3fe5e319845125f5ad369bb6' New sha1sum is: '870510fc4c6453eddfc19a7dab953d4bbcb1881e1' Old sha256sum was: '253154d4d4170a75c2ee68816ea847d7946bac5898dd1b66ed1a6ed57aa3d82c' New sha256sum is: '337915bb86691d5cc88cb645f3fa165a00155d7f621e1bcca40375f4d501c6d'
id	1775903926.196403
input.type	log
location	syscheck
manager.name	wazuh.manager
rule.description	Integrity checksum changed.

5.Wnioski i rekomendacje

5.1. Czy rozwiązanie jest skuteczne?

Przeprowadzone testy w środowisku hybrydowym jednoznacznie potwierdzają bardzo wysoką skuteczność platformy Wazuh. System udowodnił, że potrafi działać nie tylko jako pasywny monitor logów (SIEM), ale również jako aktywne narzędzie obronne (XDR/IPS). Szczególną skuteczność odnotowano w następujących obszarach:

- **Czas reakcji:** Moduł FIM z dyrektywą `realtime="yes"` wykrył modyfikację krytycznych plików (atak Defacement) w ułamku sekundy, natychmiast raportując zmianę sumy kontrolnej.
- **Autonomiczna obrona:** Wdrożenie modułu *Active Response* pozwoliło systemowi bez udziału człowieka zablokować adres IP maszyny atakującej protokół SSH. System udowodnił zdolność do przerywania trwających ataków (Brute Force) zanim osiągną one swój cel.

5.2. Trudności napotkane podczas wdrożenia

- **Specyfika działania modułu FIM:** Zauważono, że FIM opiera się ściśle na kryptograficznych funkcjach skrótu (hash). Powtórzenie ataku nadpisującego plik dokładnie tą samą treścią nie generowało nowego alertu, co wymagało modyfikacji ładunku (ang. *payload*) w celu wymuszenia reakcji systemu.
- **Rygorystyczna składnia konfiguracyjna:** Silnik Wazuh bazuje na plikach XML, które są skrajnie wrażliwe na literówki i błędy w tagach. Drobny błąd w strukturze pliku skutkował całkowitym brakiem możliwości uruchomienia usługi agenta.

5.3. Wymagania do wdrożenia w skali całej firmy (Skalowalność)

Aby przenieść przetestowane rozwiązanie z poziomu laboratorium (Proof of Concept) do środowiska produkcyjnego w dużej organizacji, konieczne jest spełnienie następujących wymagań:

- **Infrastruktura i Wysoka Dostępność (HA):** Zamiast wdrażania menedżera na pojedynczej maszynie (Single-node Docker), w firmie konieczne jest wdrożenie architektury klastrowej (Wazuh Cluster) z oddzielnymi węzłami indeksującymi (Wazuh Indexer), aby obsłużyć tysiące logów na sekundę bez przeciążenia sprzętu.
- **Stała Adresacja i DNS:** Serwery centralne (Managers) bezwzględnie muszą posiadać statyczne adresy IP i być osiągalne dla agentów za pośrednictwem

nazw domenowych (FQDN). Rozwiązuje to problem zrywanych połączeń po zmianach w konfiguracji sieci.

- **Automatyzacja wdrożeń:** W dużej firmie ręczna instalacja i konfiguracja agentów na każdym serwerze (jak miało to miejsce w laboratorium) jest nieefektywna. Konieczne jest użycie narzędzi klasy IaC (Infrastructure as Code), takich jak Ansible, Puppet lub SCCM, do masowego wdrażania agentów.